

Task 1: Real-Life Uses of Crontab

Scenario	Benefit
Automatic database backup	Protects important data without manual work.
Log file cleanup	Saves disk space by removing old logs automatically.
Security updates	Keeps the system updated and secure.

Task 2: Crontab Entry for Playlist Backup

Suppose the script is stored at:

- /home/student/update_playlist.sh

Crontab entry:

- 0 2 * * * /home/student/update_playlist.sh

Task 3: Create Backup Script

Create the script:

- mkdir -p ~/scripts
- vim ~/scripts/backup_instagram_photos.sh

Add the following content:

```
#!/bin/bash
DATE=$(date +%F)
SOURCE="/home/student/InstagramPhotos"
DEST="/home/student/Backups/InstagramPhotos_$(date +%F)"
mkdir -p "$DEST"
cp -r "$SOURCE"/* "$DEST"
echo "$(date): Backup completed successfully to $DEST"
```

Save and exit.

Make the script executable:

- chmod +x ~/scripts/backup_instagram_photos.sh

Task 4: Schedule Backup Every Sunday at 11:30 PM

Open crontab:

- `crontab -e`

Add:

- `30 23 * * 0 /home/student/scripts/backup_instagram_photos.sh >>
/home/student/backup_log.txt 2>&1`

Check crontab:

- `crontab -l`

Task 5: View Last 5 Lines of log

After the cron job runs:

- `tail -5 ~/backup_log.txt`